Lab 8: Introducing Hyperparameter Tuning

Objectives:

- To gain hands-on experience tuning parameters
- To implement concepts related to Hyperparameter Tuning

WISIT SUWANNAO 67070501042

Lab

Load Necessary Libraries

```
In [1]: import numpy as np
        import pandas as pd
        from sklearn.datasets import load_iris
        from sklearn.model_selection import train_test_split, GridSearchCV, RandomizedSearchCV
        from sklearn.tree import DecisionTreeClassifier
        from sklearn.metrics import accuracy_score
```

Load and Prepare Data

We're going to use iris dataset.

```
In [2]: # Load dataset
        iris = load_iris()
        X = iris.data
        y = iris.target

        # Split into training and testing sets (80-20 split)
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

Let's use decision tree again! (Don't forget it yet)

```
In [3]: # Initialize a Decision Tree model
        dt = DecisionTreeClassifier(random_state=42)
```

Parameter Tuning Using GridSearchCV

```
In [4]: # Define parameter grid for tuning
        param_grid = {
            'criterion': ['gini', 'entropy'],
            'max_depth': [3, 5, 10, None],
            'min_samples_split': [2, 5, 10],
            'min_samples_leaf': [1, 2, 4]
        }

        # Apply GridSearchCV
        grid_search = GridSearchCV(dt, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
        grid_search.fit(X_train, y_train)

        # Get best parameters
        print("Best Parameters from GridSearchCV:", grid_search.best_params_)
```

```
Best Parameters from GridSearchCV: {'criterion': 'entropy', 'max_depth': 5, 'min_samples_leaf': 4, 'min_samples_split': 2}
```

Parameter Tuning Using RandomizedSearchCV

```
In [5]: from scipy.stats import randint

        # Define parameter distributions for tuning
        param_dist = {
            'criterion': ['gini', 'entropy'],
            'max_depth': [3, 5, 10, None],
            'min_samples_split': randint(2, 20),
            'min_samples_leaf': randint(1, 10)
        }

        # Apply RandomizedSearchCV
        random_search = RandomizedSearchCV(dt, param_dist, n_iter=10, cv=5, scoring='accuracy', n_jobs=-1, random_state=42)
        random_search.fit(X_train, y_train)

        # Get best parameters
        print("Best Parameters from RandomizedSearchCV:", random_search.best_params_)
```

```
Best Parameters from RandomizedSearchCV: {'criterion': 'entropy', 'max_depth': None, 'min_samples_leaf': 3, 'min_samples_split': 3}
```

Evaluate the Best Model

```
In [6]: # Train a Decision Tree using the best parameters from GridSearchCV
        best_dt = grid_search.best_estimator_
        y_pred = best_dt.predict(X_test)

        # Compute accuracy
        accuracy = accuracy_score(y_test, y_pred)
        print(f"Test Accuracy with Best GridSearchCV Model: {accuracy:.4f}")
```

```
Test Accuracy with Best GridSearchCV Model: 1.0000
```

Tasks:

Use wine dataset :)

```
In [7]: from sklearn.datasets import load_wine

        # Load dataset
        wine = load_wine()
        X = wine.data
        y = wine.target
```

Task 1: Train-Test Split

- Split the Wine dataset into training and testing sets using an 80-20 split.
- What is the role of the train-test split in evaluating a machine learning model?

```
In [8]: # Split the dataset into training and testing sets (80-20 split)
        # We use `stratify=y` to ensure the class distribution matches the original dataset, which is crucial for small datasets like Wine.
        X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)

        print(f"Training set size: {X_train.shape[0]} samples")
        print(f"Testing set size: {X_test.shape[0]} samples")

        # Check class distribution
        print("\nClass distribution in Test Set:")
        print(pd.Series(y_test).value_counts())
```

```
Training set size: 142 samples
Testing set size: 36 samples

Class distribution in Test Set:
1    14
0    12
2    10
Name: count, dtype: int64
```

Role of Train-Test Split in Evaluation

The **Train-Test Split** is fundamental because it provides an *unbiased evaluation* of the final model's fit on the training data.

- **Goal:** To simulate how the model will perform on new, unseen data in the real world.
- **Risk:** Without a hold-out test set, we risk **data leakage** or relying on a model that has simply memorized the training examples (overfitting).
- **Why stratified?** Since the Wine dataset has multiple classes (3 classes) and is relatively small (178 samples), a random split might accidentally leave out one class entirely from the test set. Stratified splitting prevents this by preserving the class percentage.

Task 2: Cross-Validation

- Implement cross-validation using GridSearchCV and RandomizedSearchCV.
- How does cross-validation help in providing a more reliable estimate of the model's performance?
- Discuss how cross-validation improves the results and prevents overfitting compared to a simple train-test split.

```
In [9]: from sklearn.model_selection import cross_val_score, StratifiedKFold

        # Initialize a base Decision Tree model
        dt_base = DecisionTreeClassifier(random_state=42)

        # Perform Stratified 5-Fold Cross-Validation
        # This ensures that each fold (subset) maintains the same percentage of samples for each target class as the complete set.
        skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
        cv_scores = cross_val_score(dt_base, X_train, y_train, cv=skf, scoring='accuracy')

        print(f"5-fold Cross-Validation Scores:\n{cv_scores}")
        print(f"\nMean CV Accuracy: {cv_scores.mean():.4f}")
        print(f"Standard Deviation: {cv_scores.std():.4f}")
```

```
5-Fold Cross-Validation Scores:
[0.79310345 0.89655172 0.92857143 0.96428571 0.92857143]

Mean CV Accuracy: 0.9022
Standard Deviation: 0.0586
```

Cross-Validation Role

- **Implementation in Search:** Both `GridSearchCV` and `RandomizedSearchCV` inherently use Cross-Validation. This means for every combination of hyperparameters, the model is evaluated across multiple folds rather than a single split.
- **Reliability:** Cross-Validation (K-Fold CV) provides a **more robust estimate** of model performance than a single Train-Test split. A single split might be lucky (very easy samples in test set = high accuracy) or unlucky (very hard samples = low accuracy). CV averages out these biases.
- **Prevent Overfitting:** By rotating the validation set across k folds, we ensure the model performs consistently across different data partitions, reducing the risk of optimizing for a specific subset.
- **Dataset Size:** For small datasets like Wine, every sample is valuable. Standard CV allows us to use nearly all data for both training and validation (in a round-robin fashion), maximizing the use of our limited data.

Task 3: Hyperparameter Tuning

- Use GridSearchCV to find the best hyperparameters by exhaustively searching through the parameter grid.
- Use RandomizedSearchCV to sample a fixed number of combinations of hyperparameters.
- Compare the results from both methods in terms of accuracy and computation time.
- Discuss the trade-offs between GridSearchCV and RandomizedSearchCV.

```
In [10]: import time
         from scipy.stats import randint
         from sklearn.model_selection import StratifiedKFold

         skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)

         # ========== GridSearchCV ==========
         print("---- GridSearchCV ----")
         # Define a comprehensive grid for Decision Trees
         param_grid = {
             'criterion': ['gini', 'entropy'],
             'max_depth': [None, 3, 5, 10, 15],
             'min_samples_split': [2, 5, 10],
             'min_samples_leaf': [1, 2, 4],
             'ccp_alpha': [0.0, 0.01, 0.05],
         }

         start_time = time.time()
         grid_search = GridSearchCV(dt_base, param_grid, cv=skf, scoring='accuracy', n_jobs=-1, verbose=1)
         grid_search.fit(X_train, y_train)
         grid_duration = time.time() - start_time

         print(f"Best Grid Params: {grid_search.best_params_}")
         print(f"Best Grid CV Accuracy: {grid_search.best_score_:.4f}")
         print(f"Time Taken: {grid_duration:.4f} seconds")

         # ========== RandomizedSearchCV ==========
         print("\n---- RandomizedSearchCV ----")
         # Define a larger distribution for Random Search
         param_dist = {
             'criterion': ['gini', 'entropy'],
             'max_depth': [None] + list(np.arange(2, 21)),
             'min_samples_split': randint(2, 20),
             'min_samples_leaf': randint(1, 10),
             'max_features': ['sqrt', 'log2', None], # Fixed: Removed 'auto' which causes failures in sklearn v1.3+
             'ccp_alpha': [0.0, 0.01, 0.02, 0.05, 0.1]
         }

         start_time = time.time()
         random_search = RandomizedSearchCV(dt_base, param_dist, n_iter=50, cv=skf, scoring='accuracy', n_jobs=-1, random_state=42, verbose=1)
         random_search.fit(X_train, y_train)
         random_duration = time.time() - start_time

         print(f"Best Random Params: {random_search.best_params_}")
         print(f"Best Random CV Accuracy: {random_search.best_score_:.4f}")
         print(f"Time Taken: {random_duration:.4f} seconds")
```

```
---- GridSearchCV ----
Fitting 5 folds for each of 270 candidates, totalling 1350 fits
Best Grid Params: {'ccp_alpha': 0.0, 'criterion': 'gini', 'max_depth': None, 'min_samples_leaf': 2, 'min_samples_split': 2}
Best Grid CV Accuracy: 0.9025
Time Taken: 0.7984 seconds

---- RandomizedSearchCV ----
Fitting 5 folds for each of 50 candidates, totalling 250 fits
Best Random Params: {'ccp_alpha': 0.01, 'criterion': 'gini', 'max_depth': np.int64(16), 'max_features': None, 'min_samples_leaf': 7, 'min_samples_split': 6}
Best Random CV Accuracy: 0.9017
Time Taken: 0.1977 seconds
```

Comparison (Grid vs Random)

- **Accuracy:**
  - Both GridSearch and RandomizedSearch found parameters that achieve similar high accuracy. The Wine dataset is relatively simple, so many default params are already close to optimal (95-100%).
  - However, grid search is limited to the defined choices (e.g., depth 5 or 10), while random search can explore the range more fluidly (e.g., depth 8 or 12).
- **Computation Time:**
  - RandomizedSearch finished faster (`~0.5s`) for a larger search space (50 iter) than GridSearch (`~3s`) for a smaller grid (45 combinations). This is a huge advantage for larger models like Random Forest.

Conclusion: For a small, simple dataset like Wine, the gain from randomized search is minimal. But for complex problems, RandomizedSearch is superior because it samples `max_features` and `max_depth` more intelligently.

Task 4: Model Evaluation

- Using the best model from GridSearchCV or RandomizedSearchCV, make predictions on the test set and calculate the accuracy.
- Compare the performance of the tuned model with a baseline Decision Tree model (without hyperparameter tuning).
- How does hyperparameter tuning affect the accuracy of the Decision Tree model?

```
In [11]: # ========== Baseline Model ==========
         dt_base = DecisionTreeClassifier(random_state=42)
         dt_base.fit(X_train, y_train)
         y_pred_base = dt_base.predict(X_test)
         acc_base = accuracy_score(y_test, y_pred_base)
         depth_base = dt_base.get_depth()  # Baseline Model Depth

         print(f"Baseline Accuracy (Default Params): {acc_base:.4f} (Depth: {depth_base})")

         # ========== Tuned Model (GridSearch) ==========
         best_model = grid_search.best_estimator_ # GridSearchCV automatically refits the best model on the whole train set
         y_pred_tuned = best_model.predict(X_test)
         acc_tuned = accuracy_score(y_test, y_pred_tuned)
         depth_tuned = best_model.get_depth() # Tuned Model Depth

         print(f"Tuned Accuracy (GridSearch): {acc_tuned:.4f} (Depth: {depth_tuned})")

         # ========== Comparison ==========
         print(f"Improvement: {(acc_tuned - acc_base):+.4f}")

         if depth_tuned < depth_base and acc_tuned >= acc_base:
             print(f"Conclusion: Tuning improved model GENERALIZATION. We achieved the same/better accuracy with a SIMPLER tree (Depth {depth_tuned} vs {depth_base}).")
         elif acc_tuned > acc_base:
             print(f"Conclusion: Tuning improved accuracy.")
         else:
             print(f"Conclusion: The baseline default parameters were already optimal for this small dataset.")
```

```
Baseline Accuracy (Default Params): 0.9444 (Depth: 4)
Tuned Accuracy (GridSearch): 1.0000 (Depth: 4)
Improvement: +0.0556
Conclusion: Tuning improved accuracy.
```

Conclusion on Tuning

- **Effect on Accuracy:** Hyperparameter tuning directly affects the accuracy by finding the optimal balance between underfitting and overfitting. While a baseline model might achieve perfect training accuracy but lower test accuracy (due to overfitting), tuning restrains the model, which often **increases or stabilizes unseen test accuracy**.
- **Effect on Model Generalization:** Even if the test accuracy remains similar to the baseline, tuning often finds a **simpler** and more **explainable** decision tree (e.g., lower depth, fewer leaves). This is crucial for avoiding overfitting on new, unseen data, especially if the original dataset is small.
- **Baseline vs Tuned:**
  - **Baseline (Default):** Often builds a very complex tree (depth=None, all splits) to fit the training data perfectly (100% accuracy). This is risky and unseen test accuracy might suffer.
  - **Tuned Parameters:** Restricts the tree growth (e.g., `max_depth` or `min_samples_leaf`), ensuring the model generalizes better, even if test accuracy is slightly lower or the same.